



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5B

Nama	Nurul Naima Sabandia
NPM	2428240070
Kelas	SI5B
Topik Tugas 1	Toko Olahraga
Entitas utama	Shoes
Tautan repositori	
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	08-10-2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya] dan [Entitas], dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, [sumber-daya] menjadi buku.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama PAW2_Kuis1_SI5B_[NIM]_[Nama].pdf, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau curl di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan Win + Shift + S; macOS tekan Cmd + Shift + 4; Linux gunakan PrtSc atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal

Bab	Isi	Bukti utama
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	
Entitas utama (tunggal)	
Alamat dasar	/
Field data	
Field wajib saat POST	
Query string untuk filter	
Data awal	
Modul lain (jika ada)	

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/[sumber-daya]	Tidak	200	Ambil semua data (dengan filter bila ada)
2	GET	/[sumber-daya]/:id	Tidak	200	Ambil satu data
3	POST	/[sumber-daya]	Ya	201	Tambah data
4	PUT	/[sumber-daya]/:id	Ya	200	Ubah data
5	DELETE	/[sumber-daya]/:id	Ya	204	Hapus data

Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.

The screenshot shows a VS Code terminal window with the following commands and output:

```
PS C:\Users\arimby\SI5B_Kuis1_NurulNaimaSabandia> node -v
v24.21.0
PS C:\Users\arimby\SI5B_Kuis1_NurulNaimaSabandia> npm -v
11.19.0
PS C:\Users\arimby\SI5B_Kuis1_NurulNaimaSabandia> npm init -y
PS C:\Users\arimby\SI5B_Kuis1_NurulNaimaSabandia> npm init -y
Wrote to C:\Users\arimby\SI5B_Kuis1_NurulNaimaSabandia\package.json:

{
  "name": "si5b_kuis1_nurulnaimasabandia",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

npm notice
npm notice New major version of npm available! 11.19.0 -> 12.2.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v12.2.0
npm notice To update run: npm install -g npm@12.2.0
npm notice
PS C:\Users\arimby\SI5B_Kuis1_NurulNaimaSabandia> npm install express cors dotenv
```

The left sidebar shows the Explorer view with the following files and folders:

- SI5B_Kuis1_NurulNaimaSabandia
 - controllers
 - shoesController.js
 - middlewares
 - cekApiKey.js
 - errorHandler.js
 - logger.js
 - models
 - shoesModels.js
 - node_modules
 - routes
 - shoesRoutes.js
 - .env
 - .env.example
 - app.js
 - package-lock.json
 - package.json

The right sidebar shows the Chat view with the "Build with Agent" section.

Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

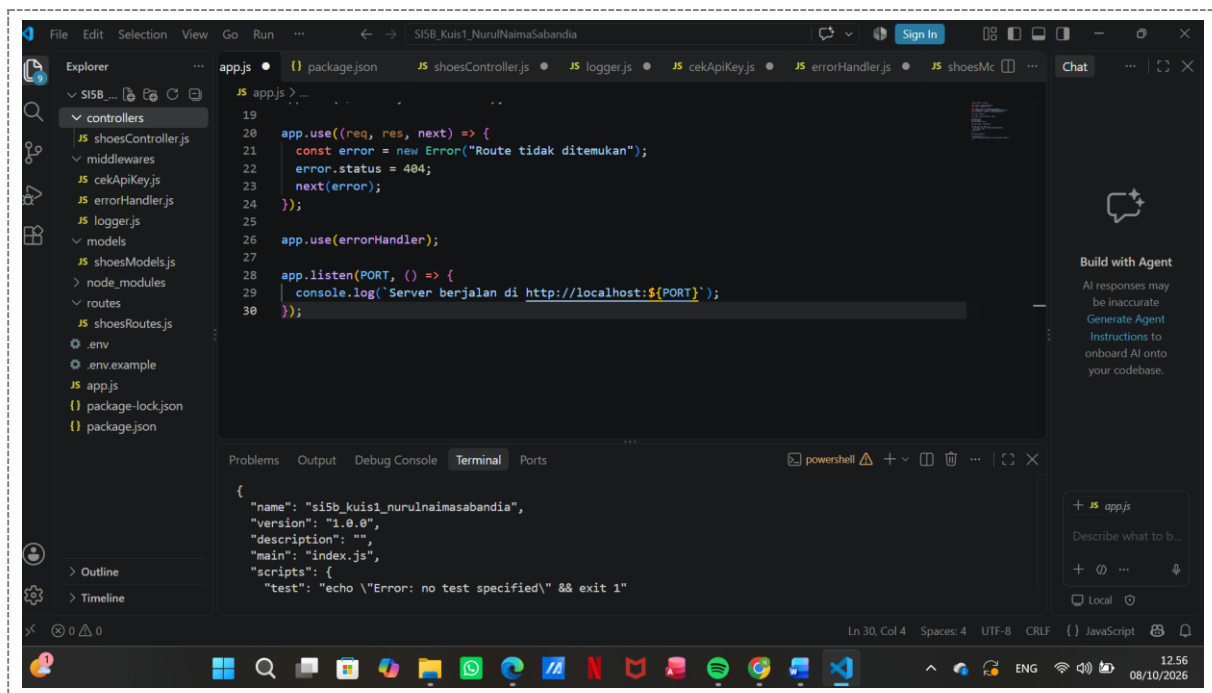
Pada tahap instalasi, digunakan Node.js sebagai runtime untuk menjalankan aplikasi backend. Perintah `npm init -y` digunakan untuk membuat file `package.json` sebagai konfigurasi project. Selanjutnya, Express.js digunakan untuk membangun RESTful API dan mengatur route serta server. CORS digunakan agar API dapat menerima request dari sumber atau aplikasi lain, sedangkan dotenv digunakan untuk membaca konfigurasi dari file `.env`, seperti PORT dan API_KEY. Versi Node.js yang digunakan perlu memenuhi persyaratan Express 5 agar aplikasi dapat berjalan dengan baik dan kompatibel dengan versi Express yang digunakan.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel *Explorer* editor atau perintah tree.



Gambar 2. Struktur folder project

Penjelasan:

Struktur folder project dibuat untuk memisahkan fungsi setiap bagian agar kode lebih terstruktur dan mudah dikelola. Folder routes berfungsi mengatur URL endpoint dan menghubungkannya dengan controller, controllers menangani request, validasi, dan response dari API, models menyimpan data sepatu serta fungsi pengolahan datanya, sedangkan middlewares berisi fungsi perantara seperti logger, pengecekan API Key, dan penanganan error.

2.2 Ringkasan tugas setiap berkas

Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

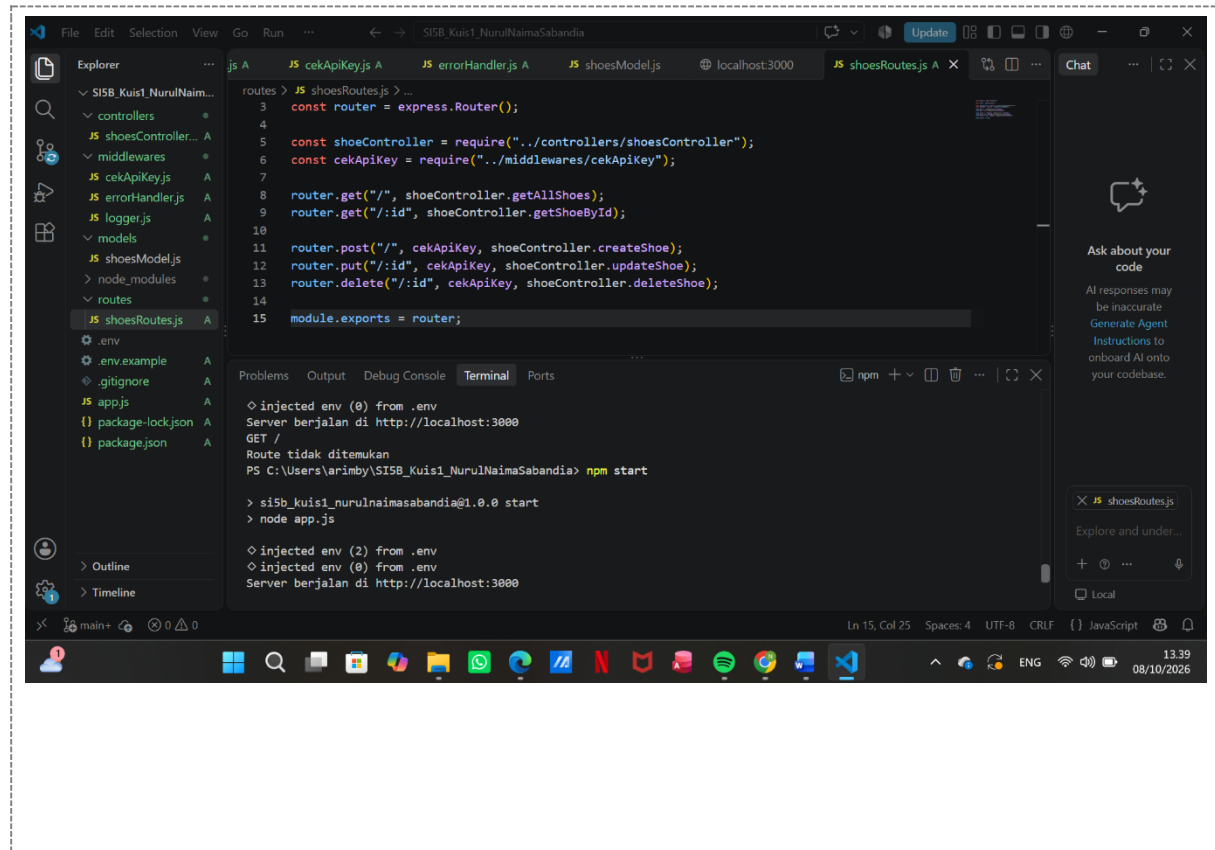
Berkas	Penjelasan Anda
models/[sumber-daya]Model.js	
controllers/[sumber-daya]Controller.js	
routes/[sumber-daya]Routes.js	

Berkas	Penjelasan Anda
middlewares/logger.js	
middlewares/cekApiKey.js	
middlewares/errorHandler.js	
app.js	

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



Gambar 3. Server berhasil dijalankan

Penjelasan:

Server dijalankan menggunakan perintah `npm start` dan menggunakan port 3000 yang ditentukan melalui konfigurasi PORT pada file `.env`. Jika port tersebut sedang digunakan oleh aplikasi lain, server tidak dapat berjalan pada port yang sama sehingga perlu menggunakan port lain.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".

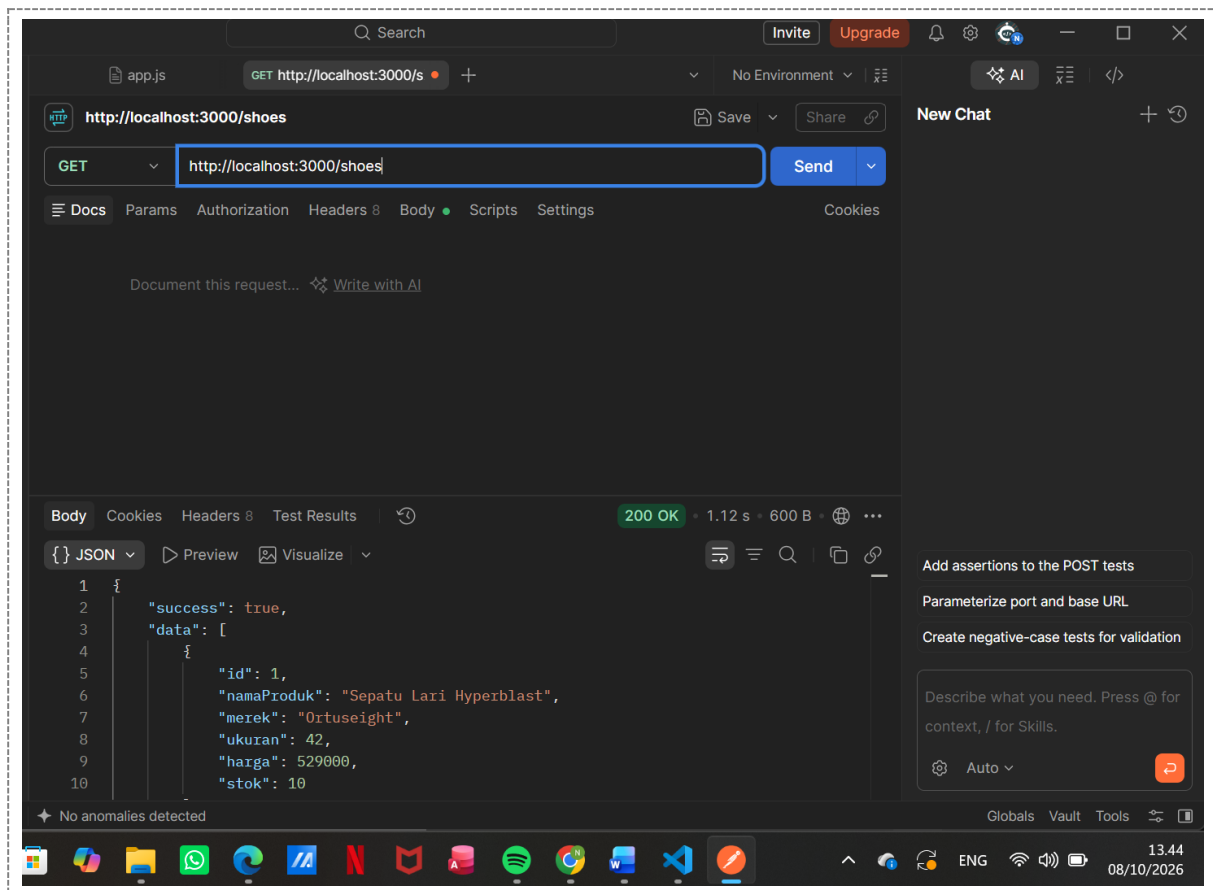
Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /[sumber-daya])

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (<i>request</i>)	GET /[sumber-daya]
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	☐ Ya ☐ Tidak



Gambar 4. Hasil GET semua data

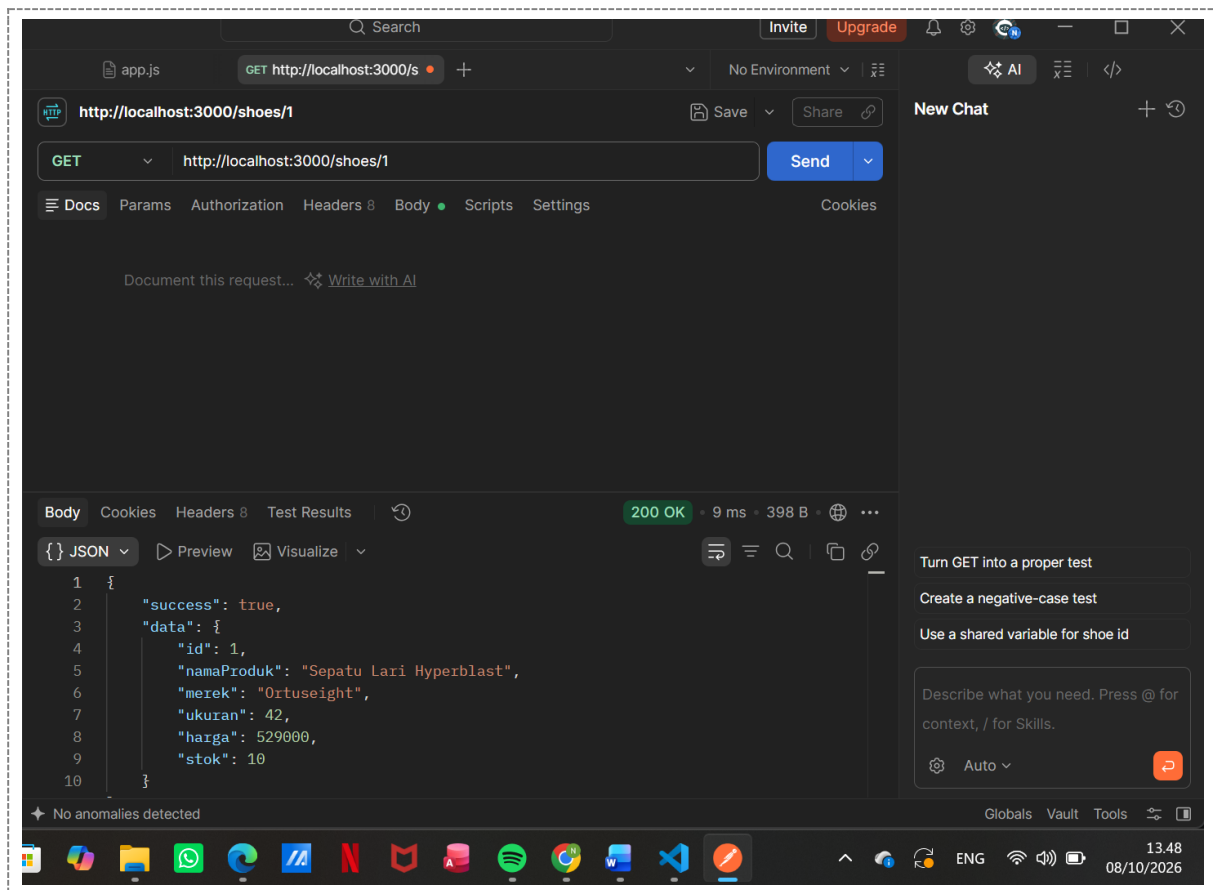
Penjelasan:

Kode status 200 OK menunjukkan bahwa permintaan berhasil diproses. Endpoint GET /shoes digunakan untuk menampilkan seluruh data sepatu. Request pertama diterima oleh Route router.get("/"), kemudian diteruskan ke fungsi Controller getAllShoes. Controller memanggil fungsi Model getAllShoes untuk mengambil seluruh data sepatu, kemudian hasilnya dikirimkan kembali dalam bentuk JSON.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (request)	GET /[sumber-harga]/[id yang ada]
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id yang diminta.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[] Ya [] Tidak



Gambar 5. Hasil GET berdasarkan ID

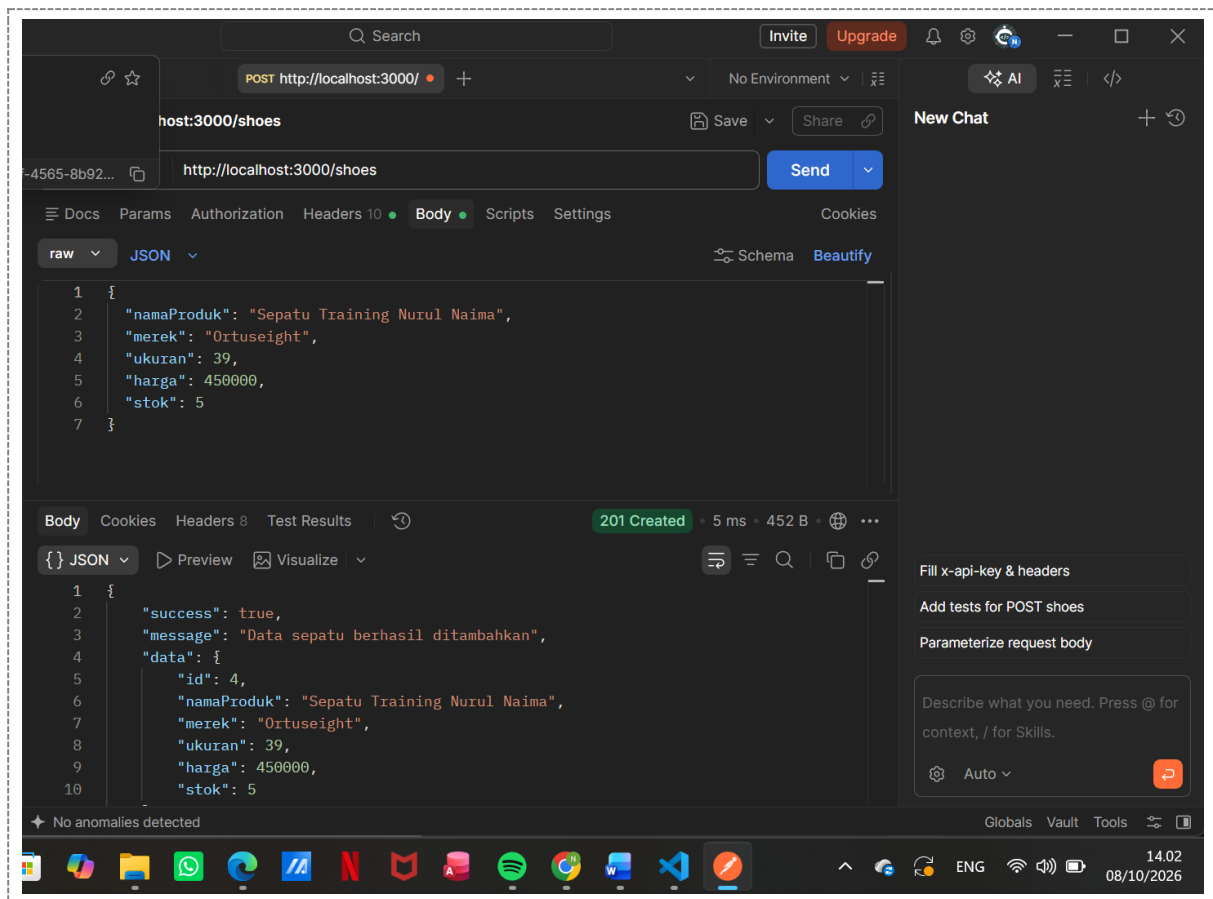
Penjelasan:

Kode status **200 OK** menunjukkan bahwa data dengan ID yang diminta berhasil ditemukan. ID diambil dari alamat menggunakan `req.params.id`, kemudian diteruskan ke Controller dan Model. ID perlu diubah menjadi angka menggunakan `Number(id)` agar dapat dibandingkan dengan nilai id yang tersimpan pada data sepatu.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header `x-api-key` dan `Content-Type`, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (request)	POST /[sumber-harga]
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "[field wajib 1]": "...", "[field wajib 2]": "..." }
Hasil yang diharapkan	201 Created; objek baru beserta id yang dibuat server.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[] Ya [] Tidak



Gambar 6. Hasil POST data baru

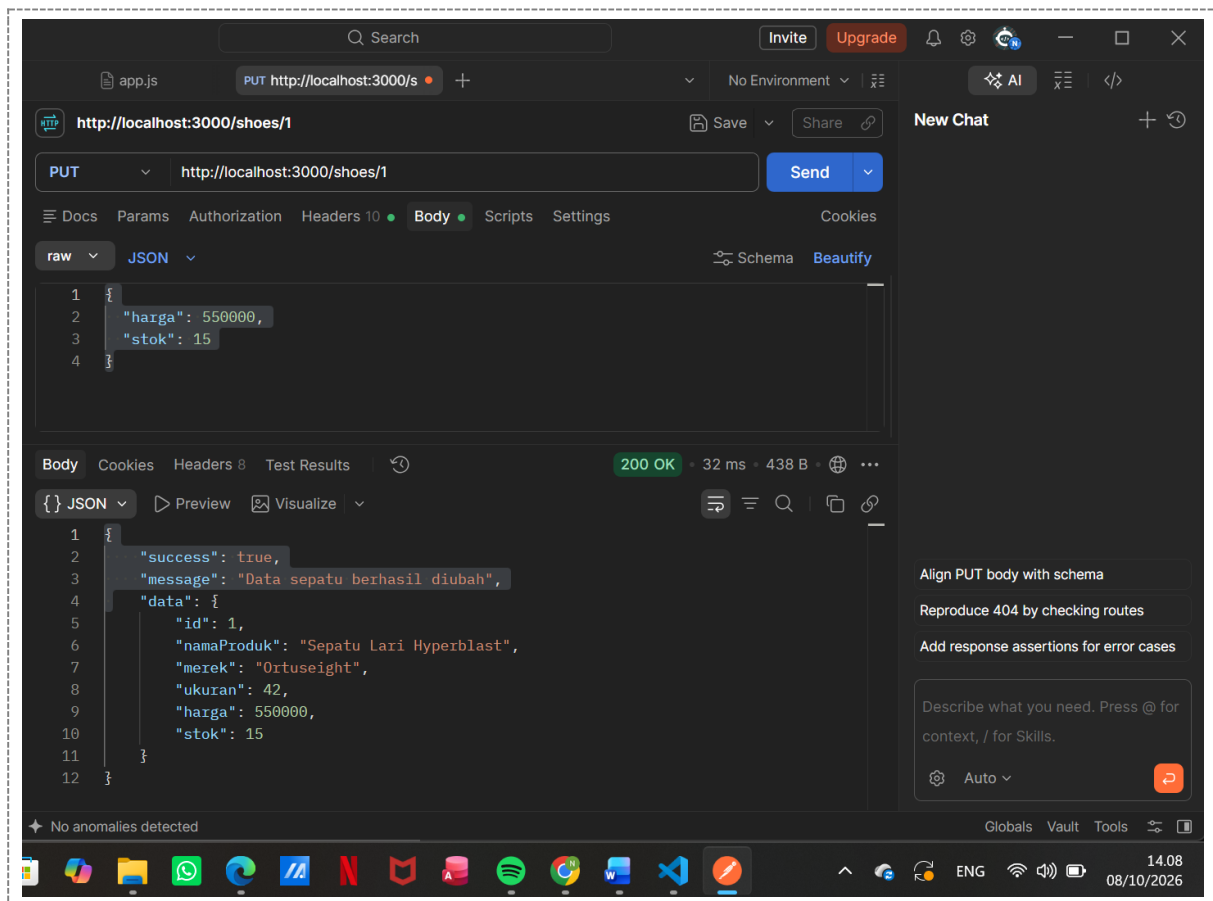
Penjelasan:

Header x-api-key digunakan sebagai pengaman endpoint POST dan diperiksa oleh middleware cekApiKey sebelum request diteruskan ke Controller. express.json() digunakan untuk membaca data JSON yang dikirim melalui body. Status 201 Created digunakan karena request berhasil membuat data sepatu baru, sehingga statusnya menunjukkan bahwa sebuah resource baru telah dibuat.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (request)	PUT /[sumber-daya]/[id data dari T03]
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "[field]": "[nilai baru]" }
Hasil yang diharapkan	200 OK; objek dengan field yang sudah berubah.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[] Ya [] Tidak



Gambar 7. Hasil PUT mengubah data

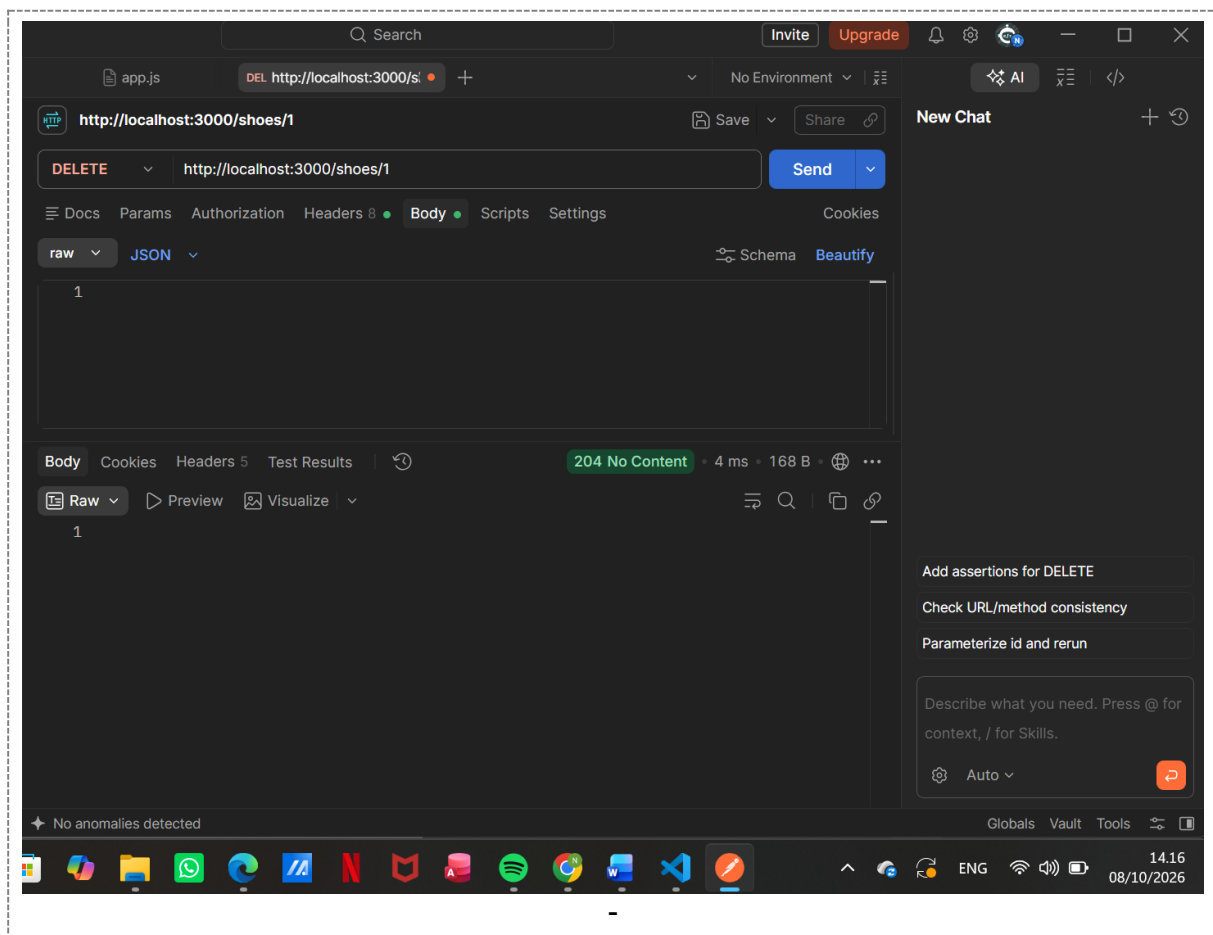
Penjelasan:

Pada pengujian ini, data sepatu dengan ID yang dipilih diubah pada field **harga dan stok**, sedangkan field lainnya tetap. Request diteruskan melalui Route ke Controller `updateShoe`, kemudian Controller memanggil Model `updateShoe` untuk mencari data berdasarkan ID dan memperbarui field yang dikirimkan. Status **200 OK** menunjukkan bahwa perubahan berhasil dilakukan.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (request)	DELETE /[sumber-daya]/[id data dari T03]
Header	x-api-key: (nilai API_KEY pada .env Anda)
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	☐ Ya ☐ Tidak



Gambar 8. Hasil DELETE data

Penjelasan:

Status 204 No Content menunjukkan bahwa data berhasil dihapus dan server tidak mengirimkan isi response. Endpoint DELETE hanya memberikan status keberhasilan karena setelah data dihapus tidak ada data yang perlu dikembalikan kepada pengguna.⁶

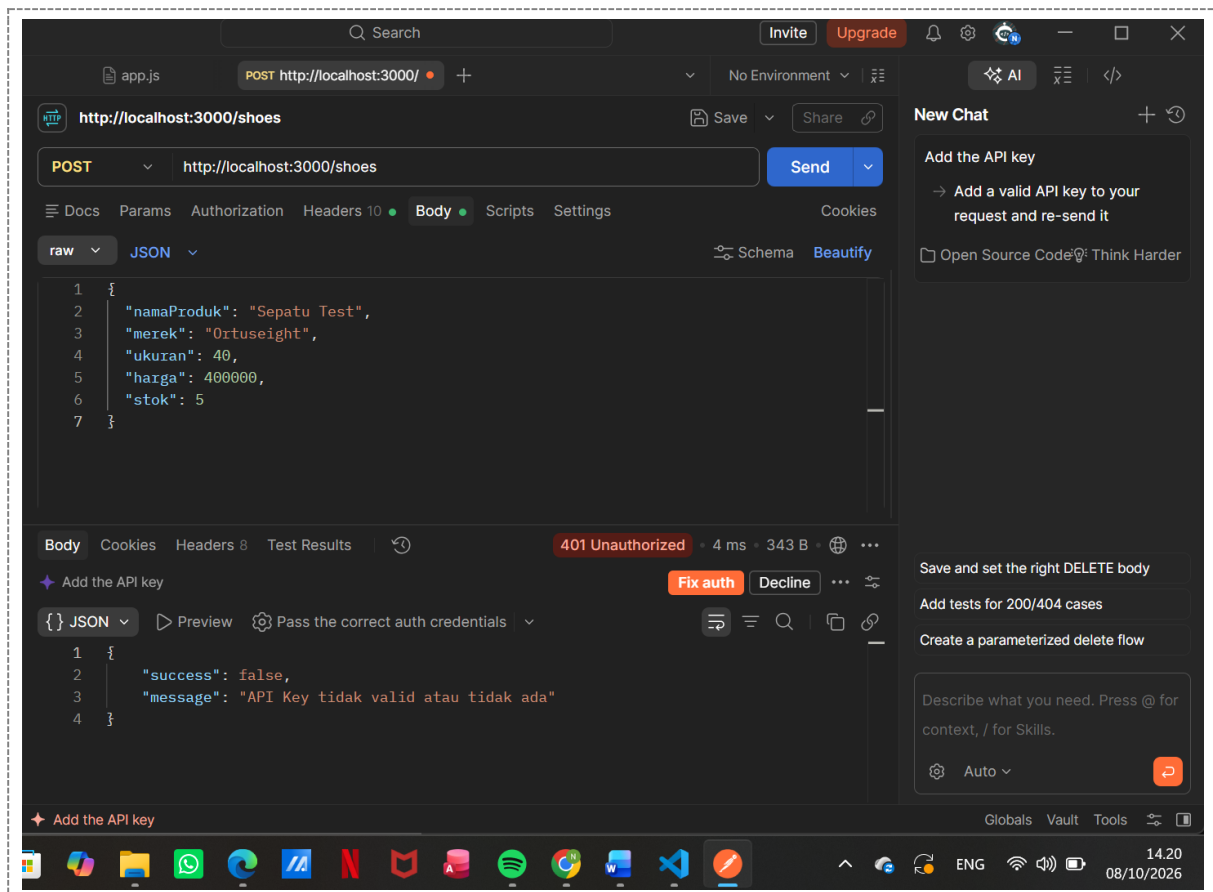
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada screenshot: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (<i>request</i>)	POST /[sumber-daya] (tanpa x-api-key)
Header	Content-Type: application/json
Body	{ data yang valid }
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	☐ Ya ☐ Tidak



Gambar 9. POST tanpa API key ditolak

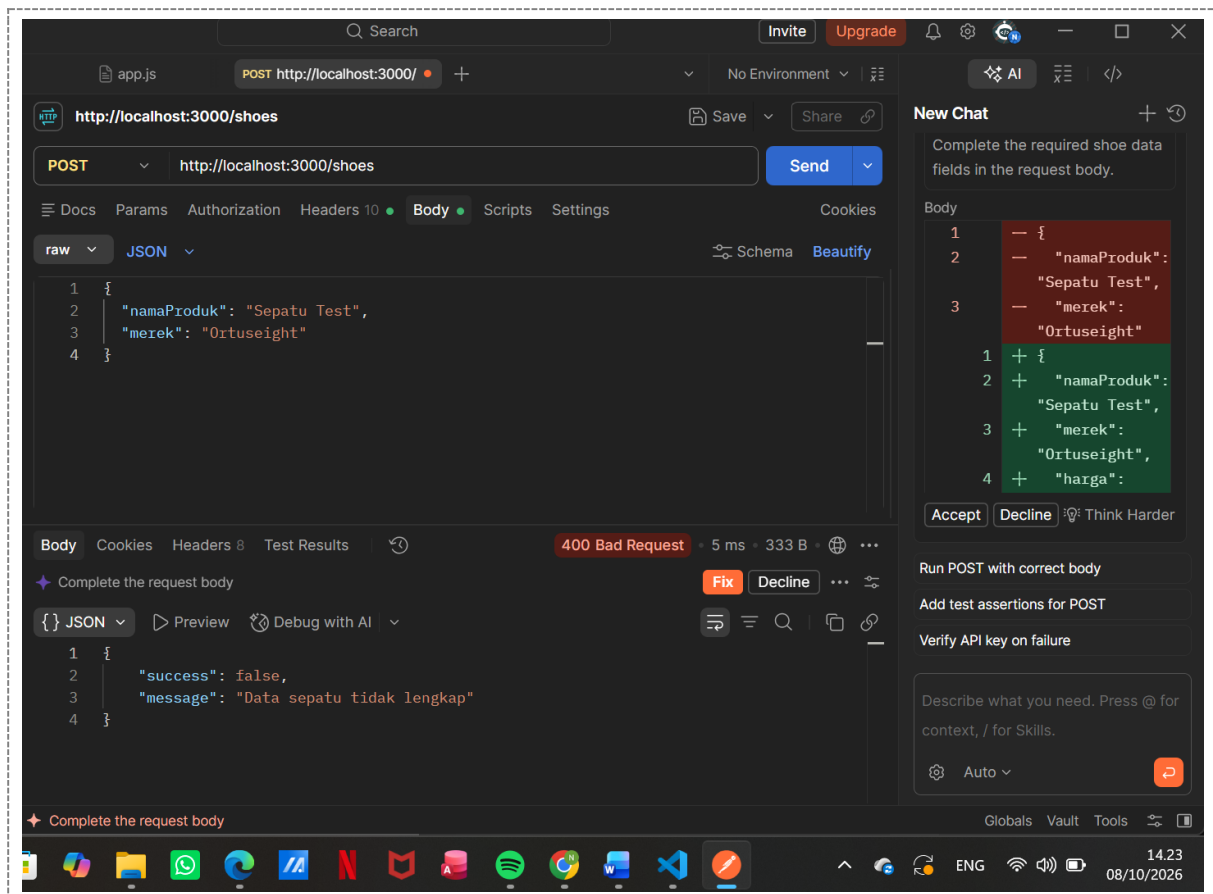
Penjelasan:

Request ditolak oleh middleware cekApiKey karena tidak terdapat x-api-key yang sesuai dengan API Key pada file .env. Karena pemeriksaan dilakukan sebelum Controller, fungsi Controller tidak sempat dijalankan. Server kemudian mengembalikan status 401 Unauthorized sebagai tanda bahwa request tidak memiliki autentikasi yang valid.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (request)	POST /[sumber-daya]
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ hanya sebagian field wajib }
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[] Ya [] Tidak



Gambar 10. POST dengan body tidak lengkap

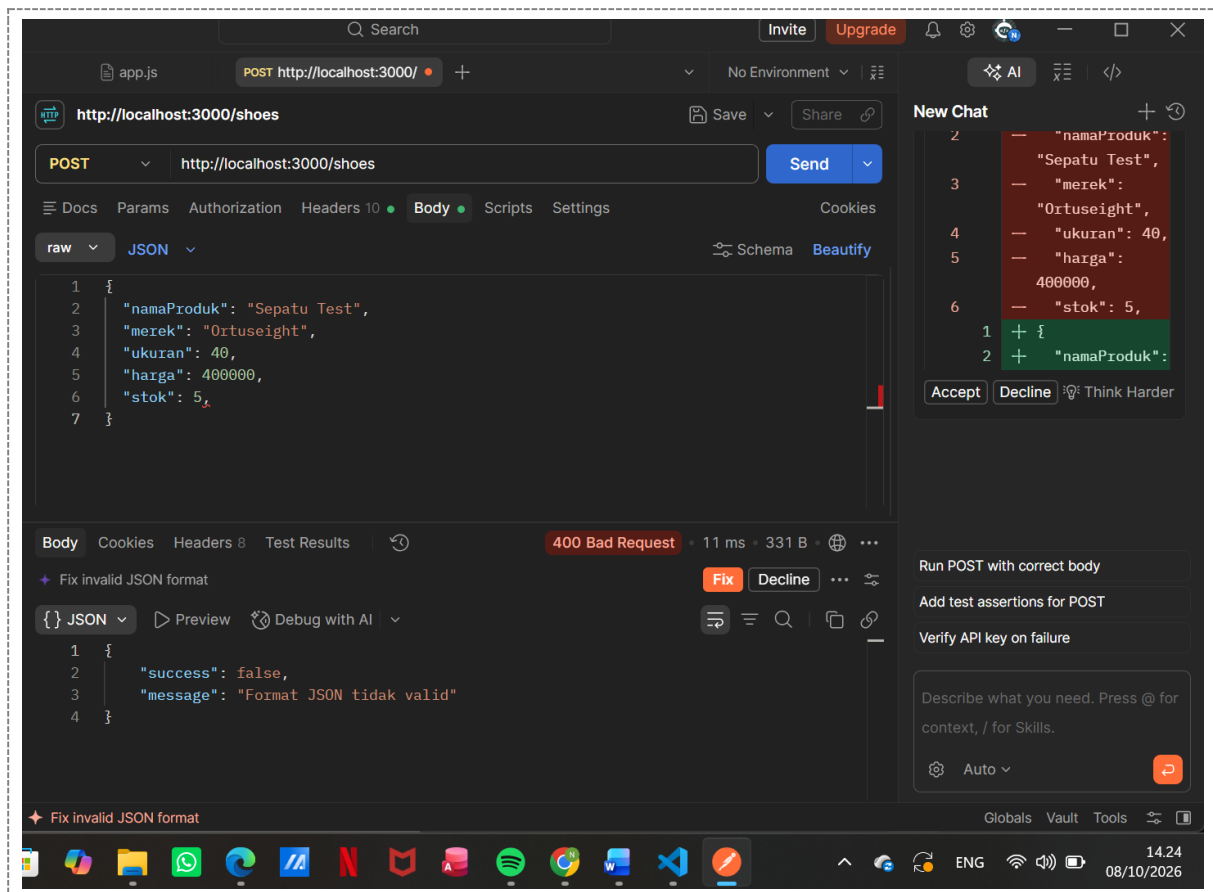
Penjelasan:

Validasi data dilakukan pada layer Controller, yaitu fungsi createShoe. Controller memeriksa field wajib seperti namaProduk, merk, ukuran, harga, dan stok. Jika terdapat field yang belum diisi, request ditolak dan server memberikan status 400 Bad Request karena data yang dikirim tidak memenuhi kebutuhan endpoint.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (request)	POST /[sumber-harga]
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ JSON dengan koma berlebih }
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML).
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[] Ya [] Tidak



Gambar 11. POST dengan JSON rusak

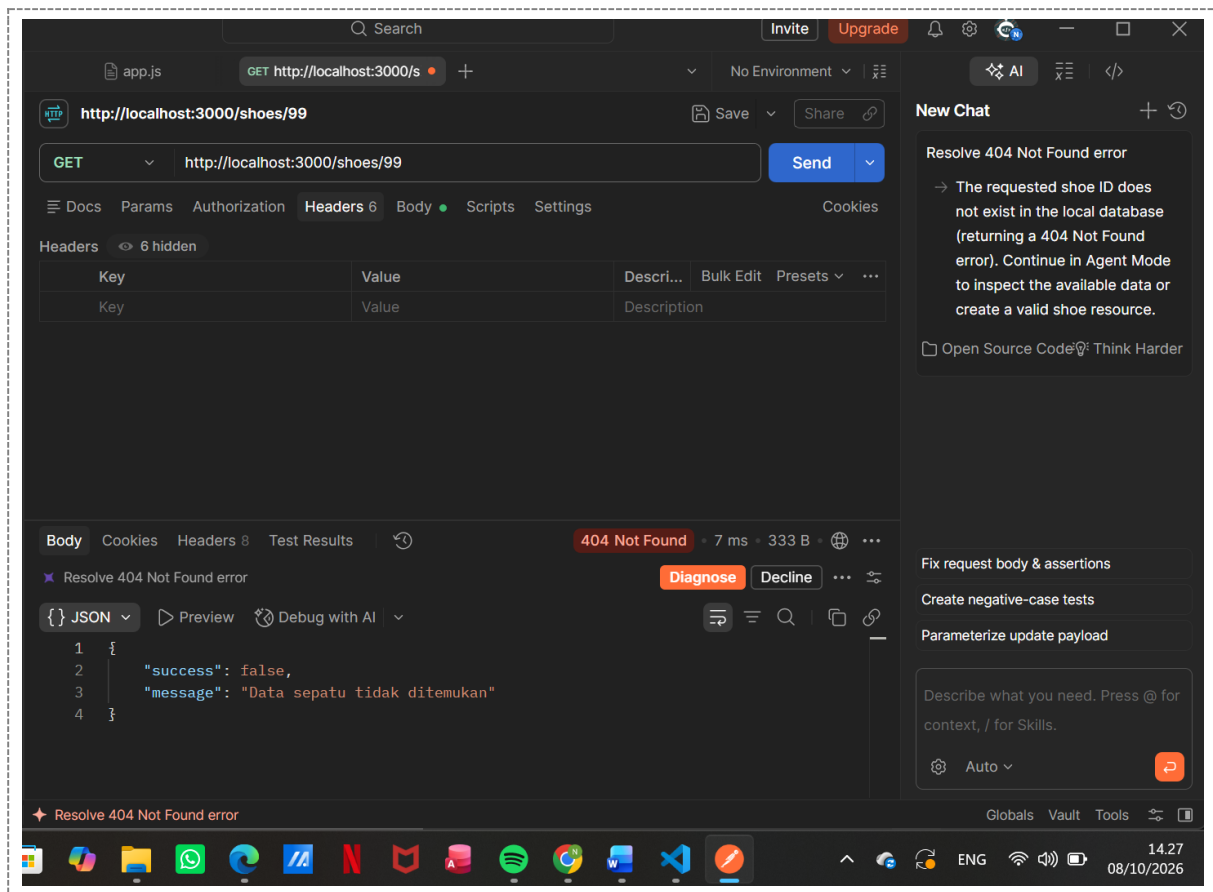
Penjelasan:

JSON yang tidak valid dideteksi ketika middleware `express.json()` mencoba membaca body request. Kesalahan tersebut kemudian diteruskan ke errorHandler, yang mengubah error menjadi response JSON dengan status 400 Bad Request dan pesan "Format JSON tidak valid".

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (request)	GET /[sumber-harga]/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[] Ya [] Tidak



Gambar 12. GET ID tidak ada

Penjelasan:

Request GET /shoes/99 mencari data dengan ID yang tidak tersedia. Route meneruskan request ke Controller getShoeById, kemudian Model mencari data berdasarkan ID tersebut. Karena data tidak ditemukan, Controller membuat error dengan status 404, lalu error diteruskan ke errorHandler untuk dikirimkan sebagai response 404 Not Found.

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200		
T02	Menampilkan satu data berdasarkan ID	200		
T03	Menambah data baru	201		
T04	Mengubah data	200		
T05	Menghapus data	204		
N01	Menambah data tanpa API key	401		
N02	Menambah data dengan body tidak lengkap	400		
N03	Mengirim JSON yang rusak	400		
N04	Mengambil ID yang tidak ada	404		

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/nurulnaimasabandia-70/si5b_kuis1_nurulnaimasabandia
Pesan commit terakhir	
Folder routes, controllers, models, middlewares ada	☐ Ya ☐ Tidak
.env tidak terunggah (ada di .gitignore)	☐ Ya ☐ Tidak
.env.example ada	☐ Ya ☐ Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

- Route berfungsi menentukan **alamat endpoint dan mengarahkan request** ke Controller yang sesuai.
- Controller berfungsi mengolah request, melakukan validasi, memanggil Model, dan memberikan response kepada pengguna.
- Model berfungsi mengelola data dan proses terhadap data, seperti mengambil, menambah, mengubah, atau menghapus data sesuai.

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

- Keuntungan error handler terpusat adalah penanganan kesalahan menjadi lebih rapi, konsisten, dan tidak perlu menulis `res.status()` berulang-ulang di setiap route. Pada kode saya, error dari Controller atau middleware diteruskan dengan `next(error)`, kemudian diproses oleh satu errorHandler.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [. . .] sudah diganti dengan topik Tugas 1 saya	✓
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	✓
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	✓
4	Identitas saya (nama atau NIM) muncul pada data uji POST	✓
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	✓
6	Tabel rekapitulasi sesuai dengan tangkapan layar	✓
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggah	✓
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	✓

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggal	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100